

Universidad de San Carlos de Guatemala

Facultad de Ingeniería

Curso: Estructuras de datos

Sección: A

Ing. Edgar Rene Ornelis Hoil

Tutor: Giovanni Saul Concohá Cax



Manual Técnico Proyecto Fase 2

Nombre: EDD MEDTRACK F2 EST

Nombre: Luis Cornelio Marroquín López

Carné: 202300727

Fecha: 23-03-2026

1. Introducción

Dentro del ámbito del desarrollo de software especializado, la aplicación de estructuras de datos avanzadas para la gestión de inventarios críticos y personal profesional es esencial para garantizar la eficiencia operativa en entornos de salud. El uso de estructuras dinámicas y jerárquicas no solo permite organizar grandes volúmenes de información de forma óptima, sino que facilita la toma de decisiones mediante una visualización clara del estado de los recursos médicos.

En este contexto, el presente proyecto consiste en el desarrollo de un sistema integral de gestión hospitalaria denominado Medtrack, desarrollado íntegramente en el lenguaje de programación Perl. La aplicación utiliza una arquitectura basada en múltiples estructuras de datos de alto rendimiento —como Árboles AVL para el personal, Árboles BST para equipos, Árboles B para suministros y una Matriz Dispersa para relaciones comerciales—, permitiendo un procesamiento de información preciso y escalable.

La solución surge como respuesta a la necesidad de contar con una herramienta técnica robusta que permita la administración de medicamentos mediante listas doblemente enlazadas y proveedores a través de listas circulares, integrando un motor de reportes visuales basado en Graphviz para documentar gráficamente la topología de cada estructura utilizada.

2. Objetivos

2.1. General

Desarrollar un sistema integral de gestión hospitalaria y control de suministros en el lenguaje Perl, que permita el procesamiento, administración y visualización de datos médicos críticos mediante el uso de estructuras de datos avanzadas para garantizar una gestión eficiente de los recursos.

2.2. Específicos

- Implementar estructuras de datos no lineales, como Árboles AVL, BST y Árboles B de orden 4, para optimizar los tiempos de búsqueda y ordenamiento de personal, equipo y suministros.
- Desarrollar una Matriz Dispersa funcional para modelar y gestionar las relaciones comerciales dinámicas entre proveedores y laboratorios fabricantes de forma eficiente en memoria.
- Mantener e integrar sistemas de almacenamiento lineal, como Listas Doblemente Enlazadas y Listas Circulares Dobles, para el control detallado de medicamentos y proveedores.
- Integrar un módulo de generación de reportes técnicos mediante Graphviz, permitiendo documentar de forma visual y organizada el estado y la topología de todas las colecciones de datos.
- Realizar la evaluación de búsquedas por código y filtros específicos sobre las estructuras para facilitar la consulta inmediata de disponibilidad y perfiles de usuario.

3. Especificación técnica

3.1. Requisitos de hardware

- Procesador: Quad-core a 2.5 GHz o superior
- RAM: 8 GB o superior
- Almacenamiento: 300 mg de espacio libre
- Monitor: Resolución 1920x1080 o superior

3.2. Requisitos de software

- Sistema operativo: Windows 10/11 (64 bits), macOS 11 o superior, o distribuciones Linux compatibles (Ubuntu, Debian, Fedora).
- Intérprete: Perl versión 5.30 o superior.
- Librerías de Interfaz: Gtk3 (Glib-Object Introspection).
- Herramientas Gráficas: Graphviz (versión más reciente) instalada y configurada en las variables de entorno del sistema.
- Formato de Intercambio: Soporte para lectura y escritura de archivos en formato JSON.

4. Descripción de la solución

La solución desarrollada consiste en un sistema de gestión hospitalaria integral construido en el lenguaje Perl, utilizando la librería Gtk3 para la interfaz gráfica. El sistema permite la administración eficiente de personal médico, suministros, equipos y medicamentos mediante el uso de estructuras de datos optimizadas que garantizan tiempos de respuesta mínimos en operaciones de búsqueda y ordenamiento.

4.1. Arquitectura del sistema

El programa opera bajo un modelo de gestión modular dividido en tres capas principales:

- **Capa de Interfaz (GUI):** Desarrollada con Gtk3, se encarga de capturar las entradas del usuario y desplegar la información. Incluye paneles diferenciados para administradores (gestión total) y usuarios (consultas de disponibilidad).
- **Capa de Lógica y Estructuras:** Es el núcleo del sistema donde residen las implementaciones de los controladores. Utiliza Árboles AVL para el personal, Árboles BST para equipos, Árboles B para suministros, una Matriz Dispersa para relaciones proveedor-fabricante y Listas Enlazadas (dobles y circulares) para inventarios.
- **Capa de Reportes:** Módulo especializado que traduce el estado de las estructuras en memoria a lenguaje de descripción de gráficos (DOT). Utiliza el motor de Graphviz para renderizar imágenes en tiempo real, permitiendo la auditoría visual del sistema.

4.2. Flujo de ejecución del programa

El flujo lógico sigue una secuencia orientada a la integridad de los datos:

- **Carga Masiva:** El sistema permite la importación de datos mediante archivos JSON. El controlador de carga procesa el archivo e inserta los objetos en sus respectivas estructuras, validando duplicados y generando códigos automáticos si es necesario.
- **Autenticación:** El módulo de login verifica las credenciales en el Árbol AVL. Dependiendo del nivel de autorización del nodo (Tipo 5 para Admin, otros para Usuario), redirige a la ventana correspondiente.
- **Gestión y Consultas:** En el modo Administrador, el flujo permite insertar, eliminar y visualizar reportes técnicos de todas las estructuras.
- **En el modo Usuario,** el sistema realiza búsquedas optimizadas por código en las estructuras (BST, B-Tree, Listas) para mostrar disponibilidad y alertas de stock bajo.
- **Generación de Reportes:** Al solicitar la visualización, el sistema recorre cada estructura, genera un archivo .dot con la topología actual y ejecuta el comando de Graphviz para producir el archivo .png que se muestra en la interfaz.

5. Lógica del programa

El sistema está estructurado bajo el paradigma de programación orientada a objetos (POO), utilizando paquetes para separar la lógica de las estructuras de datos, el modelado de entidades y la gestión de la interfaz de usuario.

5.1. Estructura de Clases principales

Paquete / Clase	Descripción
Controlador::avl	Gestiona el Árbol AVL para el personal médico. Implementa la lógica de rotaciones (simples y dobles) para mantener el factor de equilibrio y optimizar búsquedas por nombre de usuario o número de colegio.
Controlador::btree	Implementa un Árbol B de orden 4 para el inventario de suministros. Gestiona la división de nodos (split) y el almacenamiento de múltiples claves por página para un acceso eficiente a grandes volúmenes de datos.
Controlador::matrizDispersa	Controla la estructura ortogonal para las relaciones entre proveedores y fabricantes. Utiliza cabeceras en X e Y para permitir el acceso por coordenadas de empresa.
Controlador::listaDoblemente	Implementa la lógica de almacenamiento lineal para medicamentos, permitiendo recorridos bidireccionales y ordenamiento por código.
Reportes::*	Módulos especializados que recorren cada estructura y construyen la sintaxis DOT para generar los diagramas visuales a través de Graphviz.

Paquete / Clase	Descripción
Modelo:.*	Define las plantillas de datos para Usuario, Medicamento, Suministro, Proveedor y Equipo, asegurando el encapsulamiento mediante métodos <i>getter</i> y <i>setter</i> .

5.2. Flujo de Control de Datos

El flujo de información se basa en la interacción entre la memoria volátil y la visualización de resultados:

- **Entrada de Datos:** Se reciben objetos a través de formularios manuales o mediante la lectura de archivos JSON en el módulo de carga masiva.
- **Procesamiento:** Los controladores validan la integridad (evitando códigos duplicados) e insertan los objetos en la estructura correspondiente, disparando balanceos en el caso del AVL o *splits* en el Árbol B.
- **Búsqueda y Filtros:** Las peticiones del usuario (ej. buscar un medicamento o filtrar personal por departamento) invocan métodos de búsqueda recursiva en los árboles o secuencial en las listas.
- **Generación de Salida:** Al solicitar un reporte, la aplicación invoca el método `generar_reporte` del paquete correspondiente, el cual crea el archivo físico de imagen (.png) que se despliega en la interfaz.

6. Recomendaciones para el mantenimiento

Para asegurar la escalabilidad, estabilidad y correcto funcionamiento del sistema en futuras actualizaciones, se sugieren las siguientes directrices técnicas:

- Extensión de Estructuras: Para agregar una nueva entidad al sistema (ej. Citas Médicas), se debe:
 - Definir el nuevo modelo en el paquete `Modelo::*` con sus respectivos métodos de acceso.
 - Seleccionar e implementar el controlador adecuado según la complejidad de búsqueda requerida (Árbol o Lista).
 - Crear una nueva clase en el paquete Reportes que herede la lógica de generación DOT para Graphviz.
- Manejo de Imágenes: Se recomienda mantener la limpieza de la carpeta Imagenes mediante el borrado periódico de los archivos `.dot` y `.png` temporales para evitar la acumulación de archivos innecesarios de sesiones anteriores.
- Validación de Graphviz: Cualquier actualización en la ruta de instalación del motor Graphviz debe reflejarse en la variable `$dot_bin` de los módulos de reporte para evitar errores de ejecución en diferentes sistemas operativos (macOS/Windows).
- Integridad de Datos en JSON: Si se modifican los campos de los objetos (como añadir "Correo" al Usuario), se debe actualizar el controlador de carga masiva para que coincida con la nueva estructura del archivo fuente y evitar errores de referencia nula al momento de la inserción.
- Actualización de Interfaz: Para añadir nuevas funcionalidades de búsqueda en la vista de usuario, se recomienda seguir el patrón de la función privada `_crear_seccion_busqueda` en `Vista::user.pm` para mantener la uniformidad estética del panel.

7. Conclusiones

- Se logró la implementación exitosa de estructuras de datos no lineales (Árbol AVL, BST y Árbol B de orden 4), lo que permitió optimizar los tiempos de respuesta en la administración de personal y suministros, garantizando un acceso eficiente incluso con volúmenes crecientes de información.
- Se desarrolló una Matriz Dispersa funcional mediante una estructura ortogonal de nodos, logrando modelar con precisión las relaciones dinámicas entre proveedores y fabricantes sin desperdiciar memoria en celdas vacías.
- Se integraron satisfactoriamente sistemas de almacenamiento lineal a través de Listas Doblemente Enlazadas y Circulares Dobles, proporcionando un control detallado y flexible sobre el inventario de medicamentos y el registro de proveedores.
- Se implementó un módulo robusto de generación de reportes técnicos utilizando Graphviz, el cual documenta visualmente la topología de cada estructura de datos, facilitando la auditoría y verificación del estado del sistema en tiempo real.
- Se validó la correcta ejecución de búsquedas por código y filtros por atributos (departamento, especialidad y nivel de reorden), permitiendo que los distintos perfiles de usuario recuperen información específica y reciban alertas automáticas sobre la disponibilidad de recursos.